```
  afterEach(() => {
    td.reset()
  })

  it("uses the fake laoder to load a Project", () => {
    const project = new Project(1)
    const FakeLoader = td.constructor(ProjectLoader)
    project.projectLoader = new FakeLoader()
    td.when(FakeLoader.prototype.load()).thenResolve(input)
    project.load().then(() => {
      expect(project.name).toEqual("Project Runway")
      expect(project.firstTask().name).toEqual("Start Project")
      expect(project.lastTask().name).toEqual("End Project")
    })
  })

  it("can generate correct data for a loader", () => {
    const loader = new ProjectLoader(new Project(1))
    expect(loader.ajaxData().dataType).toEqual("json")
    expect(loader.ajaxData().url).toEqual("/projects/1.js")
  })

  it("can create a project and tasks from Data", () => {
    const project = new Project(1)
    project.loadFromData(input)
    expect(project.name).toEqual("Project Runway")
    expect(project.firstTask().name).toEqual("Start Project")
    expect(project.lastTask().name).toEqual("End Project")
  })
})
```

The second and third tests are all things you've seen before: you define some input data in the beforeEach method, call a method on project to load it, and then verify the load.

The first test is more complicated, and before I go over it in detail, let's talk about testdouble.js.

Using testdouble.js

testdouble.js is a test-double library written by Justin Searls and his crew at a company called Test Double. The library is very strict about the kinds of test doubles it helps you create in service of a specific pattern of testing and program design. Searls explains that in the documentation.[7]

For our purposes, the most significant part of the testdouble.js API as compared to RSpec's mock package is that testdouble.js makes it difficult to impossible to stub only one or two methods of an exiting object. That means it's hard to

7. https://github.com/testdouble/testdouble.js/blob/master/docs/2-howto-purpose.md#background